

In Depth Program Descriptions

Revision 0, 6/21/26

Introduction

This document describes a proof-of-concept system demonstrating that a program written in VB.NET and a program written in Fortran can exchange structured data and invoke each other's routines at runtime, even though the two programs are compiled by entirely different toolchains and run as separate operating-system processes. The system consists of four components: a VB.NET Windows Forms client application, a VB.NET dynamic-link library (DLL) that handles all communication details on the client side, a Fortran server console application, and a Fortran static library that handles all communication details on the server side.

The two programs communicate through a Windows Named Pipe, using the JSON-RPC 2.0 protocol to structure every message. Before describing the four components individually, this document introduces the three underpinning technologies — Windows Named Pipes, JSON, and JSON-RPC 2.0 — that a reader unfamiliar with inter-process communication or structured messaging will need to understand in order to follow the component descriptions that follow.

Windows Named Pipes and Inter-Process Communication

What is Inter-Process Communication?

When two programs need to share information or coordinate their work, they cannot simply read each other's memory — the operating system isolates each process in its own protected address space. Inter-process communication (IPC) is the umbrella term for the mechanisms the operating system provides to let separate processes exchange data safely. Common IPC mechanisms include sockets, shared memory, message queues, and pipes. This proof-of-concept uses Windows Named Pipes because they are efficient, well-suited to communication between two programs on the same machine, and do not require any network infrastructure.

What is a Windows Named Pipe?

A Windows Named Pipe is an IPC channel provided by the Windows operating system that is identified by a human-readable name rather than a numeric handle. The name follows the convention `\\.\pipe\<name>`, where `<name>` is a string agreed upon in advance by both programs. One process creates the pipe (the server side) and waits for a connection; the other process opens the pipe by the same name (the client side) and connects to it. Once connected, both sides can send and receive data through the pipe simultaneously — named pipes support full-duplex operation, meaning data can flow in both directions at the same time on the same connection.

From the perspective of the code that uses a named pipe, reading from and writing to it feels very much like reading from and writing to a file: data written at one end becomes available to read at the other end in the same order it was written. This makes named pipes straightforward to use from both VB.NET and Fortran, since both languages have well-established file I/O idioms.

The Byte-Stream Nature of a Named Pipe

A named pipe delivers a continuous stream of bytes with no inherent structure or boundaries. If the sending side writes two separate blocks of data one after the other, the receiving side has no built-in way of knowing where the first block ends and the second begins — it simply sees a sequence of bytes. This property is important because it means that the two programs must agree on a framing scheme: a convention that marks the start and end of each logical message within the byte stream.

Content-Length Message Framing

This system uses an HTTP-style Content-Length framing scheme, borrowed from the Language Server Protocol (LSP), to delimit messages within the byte stream. Before each JSON message is written to the pipe, a plain-text header is prepended. The header takes the form `Content-Length: N` followed by a blank line (where `N` is the exact number of bytes in the JSON payload that follows). The receiving side reads the header first, extracts the byte count `N`, and then reads exactly `N` more bytes, which constitute one complete JSON message. This approach means that messages of any length can be transmitted reliably — the receiver always knows exactly how many bytes to read before it has a complete message ready to parse.

JSON

What is JSON?

JSON, which stands for JavaScript Object Notation, is a lightweight, text-based format for representing structured data. Despite its name, JSON is completely language-independent — it is used as a data exchange format between programs written in any combination of languages, including VB.NET and Fortran. A JSON document is human-readable, which makes it straightforward to inspect and debug.

JSON Data Structures

JSON organizes data using two fundamental constructs. The first is an object, which is an unordered collection of named fields enclosed in curly braces. Each field consists of a name (always written as a quoted string) followed by a colon, followed by a value. For example, `{"method": "addint", "id": 1}` is a JSON object containing two fields: one named `method` whose value is the string `addint`, and one named `id` whose value is the number `1`. The second construct is an array, which is an ordered list of values enclosed in square brackets, such as `[3, 5]` for a list of two numbers or `["a", "b", "c"]` for a list of three strings.

JSON supports six value types: strings (quoted text), numbers (integer or floating-point), booleans (the literals `true` and `false`), the special value `null`, objects, and arrays. These constructs can be nested freely — an object can contain arrays as field values, and an array can contain objects as elements — which allows arbitrarily complex data structures to be represented in a single JSON document.

Why JSON is Used Here

JSON serves as the common data language between the VB.NET client and the Fortran server. Because both programs read and write JSON as plain text, neither program needs to know anything about the internal data representations of the other. The VB.NET side uses the Microsoft `StreamJsonRpc` library to produce and consume JSON automatically. The Fortran side uses the open-source `json-fortran` library for the same purpose. As long as both sides agree on the structure of the JSON messages — which is defined by the JSON-RPC 2.0 protocol described in the next section — they can exchange data correctly regardless of the differences between the two programming languages and their runtime environments.

JSON-RPC 2.0

What is Remote Procedure Call?

Remote Procedure Call (RPC) is a programming model in which one program invokes a subroutine or function that is implemented in a different program, possibly running on a different machine. From the calling program's perspective, the call looks much like any local subroutine call: the caller provides a method name and a set of parameters, and eventually receives a result. The underlying communication — packaging the call into a message, transmitting it, receiving the response, and unpacking the result — is handled automatically by the RPC framework, keeping the application code clean and focused on business logic.

The JSON-RPC 2.0 Standard

JSON-RPC 2.0 is a lightweight RPC protocol that uses JSON as its message format. It defines exactly three message types. A request is a JSON object sent by the caller that contains four fields: a `jsonrpc` field set to the string "2.0" (identifying the protocol version), a `method` field containing the name of the procedure to invoke, a `params` field containing the parameters as either a JSON array (for positional parameters) or a JSON object (for named parameters), and an `id` field containing a unique integer that the caller assigns to identify this particular call. A response is a JSON object sent back by the called program after it has executed the method; it contains the matching `id`, and either a `result` field with the return value or an `error` field with a structured description of what went wrong. A notification is a request that omits the `id` field entirely; the receiving side executes the named method but sends no response, making notifications a fire-and-forget signaling mechanism.

Standard Error Codes

The JSON-RPC 2.0 specification defines five standard error codes that a server must return when specific protocol-level failures occur. A Parse Error (code -32700) is returned when the received message is not valid JSON and cannot be parsed at all. An Invalid Request (code -32600) is returned when the message is valid JSON but does not conform to the JSON-RPC 2.0 object structure. A Method Not Found error (code -32601) is returned when the method name in a request does not match any registered handler. An Invalid Params error (code -32602) is returned when the parameters supplied do not match what the named method expects. An Internal Error (code -32603) is returned when the server encounters an unexpected condition while executing the method. Beyond these five standard codes, a server may define its own application-specific error codes for domain-level failures such as integer overflow or division by zero.

Bidirectional Communication

JSON-RPC 2.0 is a symmetric protocol — either side of a connection can send requests and notifications to the other. In this proof-of-concept the VB.NET client is the primary caller: it sends requests naming the Fortran methods it wants to invoke and receives the corresponding responses. However, the Fortran server also sends messages back to the client unprompted, in the form of JSON-RPC notifications. These server-initiated notifications carry real-time progress updates during long-running computations, acknowledgements of control signals such as pause and resume commands, and status messages. The client registers handler methods that the JSON-RPC framework calls automatically when a matching notification arrives.

Batch Requests

The JSON-RPC 2.0 specification also defines a batch mode in which a client packages multiple requests into a single JSON array and sends them all in one transmission. The server processes each request in the batch independently and assembles the individual responses into a single batch response array. This proof-of-concept exercises batch mode as part of its protocol testing, although in practice the Fortran server processes one complete framed message per cycle, so batch items are not processed concurrently.

As noted elsewhere, the implementation of batch requests for Revision 0 is not fully compliant with the JSON RPC 2.0 standard, and should be implemented in Revision 1 at a later date.

VB.NET JSON-RPC Client Program

Overview

The VB.NET JSON-RPC Client Program is a Windows Forms desktop application that provides a graphical user interface through which a user can invoke remote procedures on the Fortran server, observe results, and exercise the behaviour of the communication protocol under both normal and error conditions. It is the component of the proof-of-concept that a user interacts with directly. All low-level communication concerns — establishing the named pipe connection, framing messages, managing the JSON-RPC session, and handling disconnection events — are delegated entirely to the companion VB.NET DLL described in Section 5. The client application itself is concerned only with gathering user input, invoking the appropriate DLL functions, and displaying the results.

Architecture

The client is structured as a multi-module VB.NET Visual Studio solution. The Windows Forms user interface resides in the main form class. Supporting modules, each with a clearly bounded responsibility, handle calculation logic, matrix data handling, input validation, UI thread marshalling, and application-wide constants. The client treats the Fortran server as an opaque service: it knows the method names the server exposes and the data formats it expects, but has no knowledge of the server's internal implementation.

Scalar Arithmetic Operations

The user selects an arithmetic operation from a drop-down list, enters one or two parameters in text fields, and clicks Execute. The client validates the inputs, sends the appropriate JSON-RPC request to the Fortran server via the DLL, receives the numeric result, and independently calculates the expected answer locally to verify correctness. Integer, real (floating-point), and complex number operations are all supported. Real and complex inputs accept plain decimal, integer, or scientific notation formats. Results are verified using a hybrid floating-point comparison strategy: relative tolerance for larger values, and absolute tolerance for values near zero, ensuring accuracy across a wide range of magnitudes.

Because the JSON-RPC 2.0 standard does not define a complex number type, complex number values are encoded as strings in the format $Re \pm Imi$ (for example, "3.5+2.1i") before being passed to the DLL. The DLL transfers these strings to the server as-is; all encoding and decoding of complex number values is the responsibility of the client application.

Matrix Operations

The client populates a 4×4 input matrix with randomly generated data of the user's chosen type, which may be integer, real, complex number, logical (Boolean), or text. It then sends the matrix and the selected operation to the Fortran server as a named-parameter JSON-RPC request, receives the result matrix, validates it for correct dimensions and data type, and displays it alongside the original. The supported operations include element-wise copy, transpose, and element-wise squaring (where applicable). Each data type follows the same structured workflow: generate, validate, extract, transmit, receive, validate response, display.

Mandelbrot Benchmark

A long-running Mandelbrot set computation is dispatched to the Fortran server as a single RPC call. The Mandelbrot set is a mathematical fractal that requires iterative complex arithmetic at each point of a two-dimensional grid; it serves here as a convenient computationally intensive workload that scales predictably. While the benchmark executes on the server, the user can send pause, resume, and stop notifications to control server-side execution without disconnecting from the pipe. As the computation progresses, the server sends JSON-RPC progress notifications back to the client at each ten-percent completion milestone, and the client displays a live progress indicator. Elapsed time is reported on completion, providing a practical demonstration of long-duration RPC calls, mid-execution notification handling, and bidirectional communication.

Protocol and Error Testing

Dedicated controls allow the user to deliberately trigger each of the five standard JSON-RPC 2.0 error conditions. The Parse Error and Invalid Request tests work by sending intentionally malformed payloads directly to the pipe stream, bypassing the normal JSON-RPC framework, and then confirming that the server responds with the appropriate error code and closes or resets the connection. The Method Not Found, Invalid Parameters, and Internal Error tests operate through the normal RPC path and verify that the error code returned in the server's response matches the expected value. All error tests automatically reconnect after errors that destroy the connection, so testing can continue without manual intervention.

A batch test sends three sequential integer operations plus a notification in a single message cycle, demonstrating the multi-call packaging feature of the JSON-RPC 2.0 protocol.

Input Validation

All user inputs are validated on the client side before any request is transmitted to the server. The validation layer checks for empty fields, verifies that numeric inputs conform to the expected format including scientific notation, and for complex numbers, parses both the real and imaginary components and classifies their numeric format for consistent display. Named-parameter inputs are parsed as JSON key-value pairs with automatic type inference. Validation errors are reported immediately and no request is sent until the inputs pass all checks.

Proof-of-Concept Considerations

The client includes one deliberate simplification made in the interest of keeping the proof-of-concept straightforward: the named pipe identifier is hard-coded as a fixed string that must match the hard-coded name in the Fortran server. In a production implementation this value would be supplied via external configuration, allowing the pipe name to be changed without recompiling either program. All other aspects of the client — protocol handling, validation, error reporting, and result verification — are implemented in a manner consistent with a production-quality application.

VB.NET JSON-RPC Client Library (DLL)

Overview

The VB.NET JSON-RPC Client Library is a reusable .NET class library (DLL) that enables a VB.NET application to act as a JSON-RPC 2.0 client communicating with a compatible server over a Windows Named Pipe. It abstracts all low-level details of pipe connection management, Content-Length message framing, protocol compliance, and error handling into a set of well-defined modules, so that the consuming application can concentrate on what it wants to do rather than on how to communicate. The library is built on top of the Microsoft StreamJsonRpc library, which handles JSON serialisation, message framing, request identifier management, and protocol version enforcement; the DLL wraps StreamJsonRpc in higher-level modules that present a clean, method-neutral API to the consuming application.

Architecture

The library is organised into seven source files, each with a distinct responsibility. The AppConstants module defines operational constants used throughout the library, including the hard-coded named pipe identifier. The ConnectionModule manages the full lifecycle of the pipe connection. The RPCOperations module provides outbound call wrapper functions. The RpcTargetHandler class handles server-to-client calls and notifications. The ValidationModule provides a library of validation and utility helper functions. The ErrorTestingModule is a diagnostic utility that tests all five standard JSON-RPC error codes. The ClientInfo class is a data class that carries client information requested by the server.

Connection Management

The ConnectionModule manages the complete lifecycle of the named pipe connection. It creates and connects the NamedPipeClientStream, initialises the JsonRpc instance with the appropriate transport stack, registers a disconnect event handler, and starts the message listener. A key configuration applied at initialisation forces all incoming server messages to be dispatched on background thread-pool threads rather than the UI thread; this prevents deadlocks that would otherwise occur when server notifications arrive while the client application is waiting for a response to a request it has already sent.

The module also provides a cleanup function that disposes all three connection objects — the pipe stream, the transport layer, and the JsonRpc instance — in the correct tear-down order to avoid resource leaks. A reconnect function attempts to re-establish a lost connection up to a configurable number of times with a configurable delay between attempts, allowing the client application to recover from transient disconnections without requiring the user to restart.

Outbound RPC Operations

The RPCOperations module provides method-neutral wrapper functions for all outbound JSON-RPC calls. No server method names are embedded in the DLL itself; the consuming application supplies the method name as a string parameter on every call. This design makes the library reusable with any JSON-RPC 2.0 server, not just the Fortran server used in this proof-of-concept.

Wrapper functions are provided for integer operations taking two integer parameters and returning an integer result, for real-number operations taking one or two double-precision parameters and returning a double-precision result, for string operations taking two string parameters and returning a string result (used for complex arithmetic by encoding complex numbers as strings in the calling application), for fire-

and-forget notifications with or without a message payload, and for named-parameter requests where parameters are supplied as a key-value dictionary. All calls are subject to a configurable timeout, and a `TimeoutException` is thrown if the server does not respond within the allotted period.

Server-to-Client Calls

Because JSON-RPC 2.0 is bidirectional, the server can initiate calls to the client. The `RpcTargetHandler` class handles this server-to-client direction. At connection time an instance of this class is registered with `StreamJsonRpc` as the target object; the framework automatically dispatches incoming server calls and notifications to the matching public method on the handler by name, using the method name in the incoming JSON-RPC message. Handlers are provided for server status messages, fire-and-forget notifications, progress percentage updates during long operations, status acknowledgement notifications such as pause and resume confirmations, data receipt acknowledgements, and client information requests from the server.

Validation and Utilities

The `ValidationModule` provides a comprehensive set of validation and utility helper functions that are available both to the DLL internally and to any consuming application. These cover connection state checking, string and integer input validation, method name validation, named-parameter JSON parsing, pipe stream availability checking, JSON-RPC message framing for raw protocol tests, error code comparison, echo response verification, error data extraction from server error response objects, and batch result arithmetic verification.

Built-In Error Code Testing

The `ErrorTestingModule` is a diagnostic utility shipped as part of the DLL that provides dedicated test functions for all five standard JSON-RPC 2.0 error codes. The tests for Parse Error and Invalid Request send intentionally malformed payloads directly to the pipe stream, bypassing `StreamJsonRpc`, and verify that the server responds appropriately and closes the connection. The tests for Method Not Found, Invalid Params, and Internal Error operate through the normal RPC path and verify the error code in the server's response. All tests automatically reconnect after errors that destroy the pipe connection, so a complete error-code test run can be executed without manual intervention between tests.

Proof-of-Concept Limitation

As a proof-of-concept, the library contains one deliberate simplification: the named pipe identifier (`MyTestPipe`) is hard-coded as a constant in the `AppConstants` module. This constant must also match the hard-coded name used in the Fortran server. In a production implementation the pipe name would be supplied via external configuration rather than embedded in source code, but for a proof-of-concept the simplification is acceptable and keeps the two programs easy to run together without additional setup.

Fortran JSON-RPC Server Program

Overview

The Fortran JSON-RPC Server is a Windows console application written in Intel Fortran. It creates a Windows Named Pipe, waits for the VB.NET client to connect, and then enters a message-processing loop: it receives a framed JSON-RPC request from the client, executes the requested operation, and writes the framed JSON-RPC response back to the client. This cycle repeats until the client disconnects. The server's own source code is focused entirely on the business logic of the operations it performs — the underlying pipe I/O, JSON parsing, request validation, method dispatch, and response serialisation are all handled by the Fortran static library described in Section 7, which the server links against at build time.

Registered Methods

The server registers twenty-eight JSON-RPC methods with the static library's method registry at startup. These methods fall into six functional categories.

Integer arithmetic: the server exposes add, subtract, multiply, and divide operations on 32-bit signed integers. All four operations use a 64-bit intermediate result to detect overflow before truncating back to 32 bits, and division includes an explicit check that the divisor is not zero.

Real arithmetic: corresponding add, subtract, multiply, and divide operations on double-precision (64-bit) floating-point values. Results are rounded to two decimal places before being returned. Division again includes a zero-divisor check.

Complex arithmetic: add, subtract, multiply, and divide operations on complex numbers. Because the JSON-RPC 2.0 standard has no complex number type, complex values are transmitted as strings in the format $re\pm imi$ (for example, "3.5+2.1i"). The server parses these strings into real and imaginary components, performs the arithmetic, and returns the result as a formatted string. Complex division uses the conjugate-of-denominator formula and checks that the magnitude of the denominator is not zero.

String operations: a `sendmessage` method that accepts a string and echoes it back to verify round-trip string handling, and a `namedparameters` method that accepts a JSON object with any number of named fields, reports each key, value, and detected type to the server console, and returns a count of the parameters received.

Mandelbrot benchmark: a single method that accepts a seed value from one to ten, which scales both the grid resolution (2,000 times seed points per axis) and the maximum iteration count (5,000 times seed per point). Three features make this benchmark interactive rather than a simple fire-and-forget call: the server sends JSON-RPC progress notifications to the client at each ten-percent milestone; the client may send a pause notification at any time, causing the server to suspend computation, notify the client that it is paused, and poll the pipe at 100-millisecond intervals until a resume notification arrives; and the client may send a stop notification to abort the benchmark early, in which case the server returns immediately with a sentinel result value indicating the run was aborted. Signal detection between computation steps uses a non-blocking pipe peek so that checking for client control messages does not stall the computation loop. On completion the server returns a floating-point checksum derived from the computed escape-time values, which the client can use to verify correctness.

Matrix operations: thirteen methods covering element-wise and structural operations on 4×4 matrices across five data types. Integer, real, text, logical, and complex matrices each have a copy method and a transpose method; integer, real, and complex matrices additionally have an element-wise squaring method. All matrix methods accept a JSON params object containing the matrix data as a sixteen-element row-major array along with metadata fields specifying data type, dimensions, operation, and ordering; these fields are validated by a shared utility before any computation proceeds.

Error Handling

All methods return standard JSON-RPC 2.0 error codes on failure, covering parse errors, invalid requests, method not found, invalid parameters, and internal errors. In addition, the server defines its own application-specific error codes for domain conditions such as integer overflow, division by zero, matrix dimension mismatch, and invalid data type. The static library ensures that a well-formed, classifiable error response is always returned to the client rather than a silent failure.

Fortran JSON-RPC Static Library

Overview

The Fortran JSON-RPC Static Library (JSONRPCStaticLibraryRevA.lib) is a Windows static library written in Intel Fortran that implements the complete JSON-RPC 2.0 protocol stack over a Windows Named Pipe. It provides a self-contained framework that any Fortran application can link against to expose named methods to external client programs, receive and validate structured JSON-RPC requests, dispatch those requests to registered Fortran subroutines, and return structured responses. Linking against the library allows the server program's own source code to remain entirely focused on application logic; all protocol-level concerns are handled by the library.

Architecture

The library is compiled from two Fortran source files and is structured as six Fortran modules.

The first source file, `jsonrpc_fortran_lib.f90`, contains five modules. The `jsonrpc_types` module defines the shared data types used throughout the library, including the abstract interface that all method handler subroutines must conform to and the `method_entry` record type used to populate the method registry. The `jsonrpc_validation` module provides protocol validation routines that operate directly on raw JSON text, inspecting message structure and field types without invoking the JSON parser; it also defines all standard JSON-RPC 2.0 error codes and contains the JSON string-escaping function used when constructing response strings. The `jsonrpc_utils` module provides low-level transport helpers including timestamp generation, HTTP-style header scanning, and Content-Length value extraction. The `jsonrpc_server_state` module holds the two shared runtime state variables — the active pipe handle and a per-request notification flag — and exposes accessor subroutines so that handler code can read and update this state without direct module coupling. The `jsonrpc_protocol` module is the core dispatch engine: it manages the method registry, parses inbound JSON messages using the `json-fortran` library, validates each field against the JSON-RPC 2.0 specification, locates the matching registered handler, invokes it, and serialises the response.

The second source file, `ServerComm.f90`, contains the sixth module: `ServerComm`. This module implements the named pipe transport layer. It provides the main blocking read loop that reads Content-Length-framed messages from the pipe, classifies each message as a single request, a notification, or a batch, dispatches through `jsonrpc_protocol`, and writes the framed response back to the pipe. It also provides helper functions for sending error responses and real-time progress notifications independently of the normal request-response cycle.

Method Registration

The library maintains an in-memory registry of up to one hundred named methods. Each entry in the registry pairs a method name string with a procedure pointer to a Fortran subroutine that conforms to the standard handler interface defined in `jsonrpc_types`. Methods are registered by the application at startup by calling into the library's registration API. On each inbound request, the protocol engine performs a case-sensitive lookup of the method name in the registry to locate the appropriate handler.

Protocol Validation

Every inbound message is fully validated before handler dispatch is attempted. The `jsonrpc` version string must be exactly "2.0", the method field must be a non-empty string, the params field must be an

array or object when present, and the id field must be an integer. Any violation produces the appropriate standard JSON-RPC 2.0 error response. The validation is performed on the raw JSON text by `jsonrpc_validation` before the more expensive JSON parsing step, so malformed messages are rejected quickly without consuming unnecessary resources.

Batch Request Support

A single inbound message may be a JSON array containing multiple individual requests. The library detects this case, iterates over each item in the array, processes each item independently through the normal validation and dispatch path, and assembles all non-empty responses into a single batch response array that is returned to the client in one framed write. If every item in the batch is a notification — meaning none of the items carries an id field — no response is sent at all, in accordance with the JSON-RPC 2.0 specification.

Notification Handling

Requests that omit the id field are treated as notifications. The registered handler subroutine is invoked normally, but the library suppresses the outbound response, ensuring the client is not sent an unsolicited reply. A per-request flag in `jsonrpc_server_state` allows progress notifications sent by a handler to be automatically suppressed when the current request is itself a notification, preventing the library from sending server-to-client notifications in response to client-to-server notifications.

Progress Notifications

Handler subroutines for long-running operations can send real-time progress updates to the client as JSON-RPC notification messages during execution, without waiting for the method to complete. The `ServerComm` module provides a dedicated helper function for this purpose. When the server's Mandelbrot benchmark handler calls this function at each ten-percent milestone, the library assembles a correctly framed JSON-RPC notification and writes it to the pipe immediately, allowing the client to update its progress display in real time.

Content-Length Framing

All messages sent and received over the named pipe are prefixed with a Content-Length: N header followed by a blank line, then the JSON body. The `jsonrpc_utils` module provides the scanning and extraction functions used to read this header from incoming messages, and the `ServerComm` module applies the same framing to all outgoing messages. This consistent framing ensures that messages of any length are delimited unambiguously within the pipe's byte stream, and that the client library on the VB.NET side can always read exactly one complete message at a time.

Structured Error Responses

All error conditions — parse failures, invalid requests, unknown method names, and handler-reported errors — produce well-formed JSON-RPC 2.0 error response objects containing a standard numeric code and a descriptive message string. String-building fallback paths in the library ensure that a valid error response is always returned even in cases where the json-fortran serialisation layer cannot be used, such as when the error occurs before the JSON parser has been successfully initialised.

Dependencies

The library has two external dependencies. The json-fortran 9.2.0 library provides all JSON parsing and serialization functionality. The Intel oneAPI HPC Toolkit provides the `ifwin` and `ifwinty` compiler modules

that supply the Win32 API declarations needed to call Windows operating system functions for pipe I/O and non-blocking peek operations. Because the library relies on Windows Named Pipes and Win32 API calls, it targets the Windows platform exclusively.

Conclusion

This proof-of-concept demonstrates that a VB.NET program and a Fortran program can successfully interoperate as a client and server using standard, well-documented technologies. The Windows Named Pipe provides a fast and reliable local communication channel that requires no network infrastructure. Content-Length framing gives both sides a deterministic way to read complete messages from the pipe's byte stream. JSON provides a common, human-readable data format that is equally accessible from both the .NET and Fortran ecosystems. The JSON-RPC 2.0 protocol imposes a clean, well-specified structure on every message, covering the full range of interaction patterns needed by this system: request-response calls, fire-and-forget notifications, server-initiated callbacks, batch operations, and structured error reporting.

The four-component architecture keeps each program's concerns cleanly separated. The two library components — the VB.NET DLL and the Fortran static library — encapsulate all protocol and transport complexity, leaving the two application components — the VB.NET client and the Fortran server — free to focus on the operations they perform. The same DLL could be used with a different server, and the same static library could be used in a different Fortran application, simply by registering a different set of methods. This separation of concerns, combined with the language-neutral JSON-RPC 2.0 standard as the communication contract, is the key design principle that makes the interoperability work.